



**Stackelberg Game Based Hybrid
Charging Solution:
Simulation Setup and Technical
Documentation**

Version 2.0

Joy Dutta

March 5, 2026

Contents

1	Introduction	2
1.1	Software prerequisites	3
2	Initial Setup	3
2.1	Simulation Parameters	3
2.2	EV Characteristics	5
3	Real World Considerations	6
3.1	Realistic Simulation Setup	6
4	Cost Functions in the Stackelberg Game	8
4.1	EV Cost Function	8
4.2	Leader's Profit Function	9
5	EV Request Handling	9
5.1	Handling Process	9
6	MCV Behavior and Dynamic Deployment	11
6.1	Scenario based Deployment	11
7	Dynamic Deployment Logic	12
7.1	Triggering Conditions	12
7.2	Decision Flow	14
8	Example Execution	15
8.1	Simulation Initialization	15
8.2	Real Time Request Handling	16
8.3	Dynamic Deployment Example	16
9	Understand Implementation	17
9.1	High Level Workflow of the Simulation	17
9.2	Function by Function Breakdown	19
9.3	Simulation Start to End: Overview	22
9.4	Simulation at a Glance	22

List of Code Listings

1 Introduction

This document describes the implementation of the Python script `simulation36_Stackelberg_Simulation_peak_hour_V8_FCS_3_MCV_6_OMEN_Greedy_Comparison_CaseA.py`. The script simulates an urban electric vehicle (EV) charging system over an eight hour evening period in a 20×20 km area with three fixed charging stations (FCSs) and up to six mobile charging vehicles (MCVs). The interaction between the infrastructure owner (leader) and EV drivers (followers) is modeled as a Stackelberg game, where the leader sets MCV prices and deployment decisions and EVs respond by minimizing their generalized charging cost.

The code supports several experimental modes through simple configuration flags:

- **CASE A (Stackelberg reference)**: original multi trigger MCV deployment and fully rational EV choice.
- **CASE B (Leader only greedy)**: greedy MCV deployment with rational EVs.
- **CASE C (EV only greedy)**: original deployment logic with myopic EV choice.
- **CASE D (Fully greedy)**: greedy MCV deployment and myopic EV choice.
- **CASE E (FCS only)**: all demand must be served by FCSs with no MCVs.

The primary focus for reproducing the Stackelberg based results is CASE A. All other cases are obtained by changing a small number of global configuration variables in the same script. This document is written so that reviewers and readers can:

1. Understand how the main components of the simulation map to the Python code.
2. Know exactly which configuration variables and CSV files they need to change to reproduce or extend a scenario.
3. Trace how the Stackelberg cost functions and deployment policies are implemented and compared against greedy baselines.

1.1 Software prerequisites

To run the simulation and reproduce the experiments, the following software environment is assumed:

- **Python:** Version 3.9 or later.
- **Python packages:** numpy, pandas, simpy, matplotlib. These can be installed via `pip install numpy pandas simpy matplotlib`.
- **Video encoding tool:** ffmpeg must be installed and available on the system PATH so that matplotlib can create MP4 animations through `FFmpegWriter`. If ffmpeg is not available, the user can comment out the video generation block in the script while still reproducing all numerical results.

2 Initial Setup

2.1 Simulation Parameters

The core parameters are defined as global variables near the top of the script. They should be treated as the first place to change when configuring a new scenario.

- **Simulation Area and Horizon**
 - Area: 20×20 km square urban region (`area_size = 20`).
 - Duration: 8 hours, discretized into minutes: `simulation_duration=8*60`.
- **Fixed Charging Stations (FCSs)**
 - Baseline count: `num_fcs = 3` from the initial FCS CSV.
 - Ports per station: `c_fcs = 10` simultaneous charging ports.
 - Charging rate: `charging_rate_fcs = 100` kWh per hour.
 - Energy buffer: `capacity_fcs = 1000` kWh per FCS (used for accounting, not as a hard capacity limit in this version).
 - Fixed price per kWh: `fixed_charging_price = 0.3`.
- **Mobile Charging Vehicles (MCVs)**

- Maximum available MCVs: $\text{max_num_mcv} = 6$.
- Ports per MCV: $\text{c_mcv} = 4$ simultaneous EVs.
- Battery capacity: $\text{battery_capacity_mcv} = 900$ kWh.
- Charging rate: $\text{charging_rate_mcv} = 150$ kWh per hour.
- Minimum usable state of charge: $\text{minimum_capacity_mcv} = 50$ kWh. Below this threshold the MCV is not allowed to commit to additional EVs.
- Deployment cost: $\text{deployment_cost_mcv} = 20$ (per deployment).
- MCV travel cost: $\text{cost_per_km_mcv} = 2$ per km of travel.
- Travel speed of MCVs: travel time is computed as road distance divided by 40 km/h and then converted to minutes when an MCV moves.

- **EV Level Parameters**

- Average energy demand: $\text{average_ev_demand} = 50$ kWh.
- Realized energy demand: sampled uniformly between 30 and 70 kWh for each requesting EV.
- EV travel cost to any charging option: $\text{cost_per_km_ev} = 0.3$ per km.
- Maximum waiting time at FCS: $\text{max_wait_time_fcs} = 1.5$ hours (90 minutes).
- Maximum waiting time at MCV: $\text{max_wait_time_mcv} = 1.0$ hour (60 minutes).
- Cost per hour of waiting: $\text{cost_per_hour} = 10$.

- **Cost Function Weights**

- zeta: weight on charging cost (default = 1).
- delta: weight on travel cost (default = 1).
- epsilon: weight on waiting time cost (default = 1).

- **Geometry and Road Network Approximation**

- Road distance factor: `road_distance_factor = 1.3` is applied to Euclidean distances to emulate detours on realistic road networks.
- Depot location: set to the center of the area (10,10) via `depot_location=(area_size/2, area_size/2)`.
- **Optional Extra FCSs**
 - `ADDITIONAL_FCS_COUNT` controls how many additional FCSs are created by promoting top ranked meeting points to FCSs.
 - Default is `ADDITIONAL_FCS_COUNT = 0` (baseline with exactly three FCSs).
 - When set to $k > 0$, the top k meeting points (by `Final_Rank` in the meeting point CSV) are converted into additional FCSs and appended to `fcs_initial_df`.

2.2 EV Characteristics

EVs are generated and handled with the following assumptions.

- **Energy demand:** Each EV has an energy requirement sampled uniformly from the interval [30, 70] kWh.
- **Location:** At arrival, the EV's spatial location is sampled uniformly over the 20×20 km area.
- **Waiting time constraints:**
 - Maximum waiting time at an FCS: 90 minutes.
 - Maximum waiting time at an MCV: 60 minutes.
- **Decision making:**
 - Under Stackelberg mode, EVs minimize a generalized cost that combines charging cost, travel cost, and expected waiting time.
 - Under greedy EV mode, EVs ignore waiting time in the decision and minimize only travel cost plus charging cost.

3 Real World Considerations

3.1 Realistic Simulation Setup

The simulation attempts to retain key features of realistic urban operation.

- **Time varying demand via Poisson arrivals:**
 - Each minute, the number of new EV requests is sampled from a Poisson distribution whose rate depends on current simulation time.
 - The function `get_arrival_rate(current_time)` returns:
 - * 1.5 EVs per minute for minutes 0 to 179 (peak).
 - * 0.5 EVs per minute for minutes 180 to 299 (normal).
 - * 0.3 EVs per minute for minutes 300 to 479 (off peak).
 - This produces a realistic evening pattern with a strong early peak and gradual decline.
- **Spatial layout from data files:**
 - FCS locations and their initial EV allocations are read from `InitialSetup_FCS.csv`.
 - Candidate meeting points for MCVs (and potential extra FCSs) are read from `InitialSetup_MeetingPoints_Updated.csv`.
 - The road network is approximated by scaling Euclidean distance with a factor of 1.3 to reflect detours and non direct routes.
- **Dynamic pricing for MCVs:**
 - Each meeting point has a price per kWh taken from the column `Dynamic_Price_per_kWh` in `InitialSetup_MeetingPoints_Updated.csv`.
 - Default synthetic meeting points (used when the CSV is missing) are assigned prices in the range 0.33 to 0.60 currency units per kWh.
 - When a new MCV is considered for deployment at a meeting point, the meeting point's price is used as the MCV price at that location,

bounded relative to the FCS price by design parameters such as θ if needed.

- **Battery SOC management for MCVs:**

- Every time an EV is served by an MCV, the MCV's state of charge decreases by the EV's energy demand.
- If a prospective EV would push the MCV SOC below `minimum_capacity_mcv`, that EV is not considered feasible for that MCV.
- MCVs with SOC below the threshold or sustained low utilization are marked for return to the depot where they can recharge and later be redeployed.

- **Travel speeds and congestion effects:**

- All MCV travel times are computed as distance/40 km per hour, converted to minutes.
- EV travel time is not explicitly modeled in the SimPy processes but its monetary cost is accounted for in the EV cost function.
- Congestion is implicit, captured through queueing and waiting time at FCSs and MCVs, rather than variable travel speeds.

- **External data files and folder structure:**

- All input configuration files are expected under the folder `Stacke1bergSimpleSimulation`:
 - * `InitialSetup_FCS.csv` with columns: `FCS_ID`, `Location_X`, `Location_Y`, `EVs_Assigned`.
 - * `InitialSetup_MeetingPoints_Updated.csv` with columns including at least: `Meeting_Point_ID`, `Location_X`, `Location_Y`, `Dynamic_Price_per_kWh`, `Final_Rank`, and optionally additional ranking features.
 - * `InitialSetup_MCV.csv` with columns: `MCV_ID`, `Meeting_Point_ID`, `Location_X`, `Location_Y`, `EVs_Assigned`, `EVs_Being_Served`, `EVs_In_Queue`.
 - * `Optional_EV_Data.csv` with columns: `EV_ID`, `Energy_Demand_kWh`, `Location_X`, `Location_Y`. When present, this file is used to

reconstruct EVs that are already allocated to MCVs at time zero. This is an *input* file residing in the initial setup folder.

- The function `robust_read_csv` is used to load these files and to fall back to default DataFrames if files are missing, empty, or malformed.
- For reproducing the published experiments, the GitHub repository should provide the exact CSV versions used for the study; changing any of these files effectively defines a new scenario.
- A separate `EV_Data.csv` is also written as *output* in the run specific output folder and contains per EV simulation results (see Section 9.2). The input and output files share a name but live in different folders and have different schemas.

4 Cost Functions in the Stackelberg Game

4.1 EV Cost Function

For each EV, the generalized cost of choosing a given charging option is modeled as

$$C_{EV} = \zeta \cdot C_{\text{charging}} + \delta \cdot C_{\text{travel}} + \epsilon \cdot C_{\text{wait}}. \quad (1)$$

The three components are:

- $C_{\text{charging}} = q \cdot p$ where q is the EV's energy demand in kWh and p is the price per kWh (either the FCS price or an MCV price at a meeting point).
- $C_{\text{travel}} = d \cdot c_{\text{km}}$ where d is the road distance between the EV location and the station or meeting point, and c_{km} is the EV's cost per km.
- $C_{\text{wait}} = t_{\text{wait}} \cdot c_{\text{hour}}$ where t_{wait} is the expected waiting time in hours and c_{hour} is the cost per hour of waiting.

The weights ζ , δ , and ϵ allow the decision logic to emphasize either price, distance, or waiting time. In all default experiments these are set to 1, so the EV minimizes the unweighted sum of the three components under the Stackelberg cases.

The flag `USE_GREEDY_EV_SELECTION` changes the decision rule:

- `USE_GREEDY_EV_SELECTION = False` (Stackelberg consistent): the EV minimizes the full generalized cost C_{EV} .
- `USE_GREEDY_EV_SELECTION = True` (greedy EV behavior): the EV minimizes only $C_{\text{charging}} + C_{\text{travel}}$ and ignores the waiting time term.

4.2 Leader's Profit Function

The charging service provider's profit aggregates revenues from both FCSs and MCVs and subtracts MCV related costs:

$$\text{Profit} = R_{\text{MCVs}} + R_{\text{FCSs}} - C_{\text{deployment}} - C_{\text{travel}}. \quad (2)$$

- R_{MCVs} is the sum of $p_m q_k$ over all EVs served by MCV m , where p_m is the price per kWh at that MCV's meeting point and q_k is the EV's energy demand.
- R_{FCSs} is the sum of $p_{\text{fcs}} q_k$ over all EVs served at FCSs, with $p_{\text{fcs}} = \text{fixed_charging_price}$.
- $C_{\text{deployment}}$ is the total cost of deploying MCVs, computed as the number of deployed MCVs times `deployment_cost_mcv`.
- C_{travel} is the sum of MCV travel costs, computed from depot to meeting points using `cost_per_km_mcv` and road distance.

The function `calculate_leaders_profit(results)` traverses the simulation results and returns the overall profit as well as the revenue and cost components. This is the main metric used for comparing Stackelberg and greedy configurations.

5 EV Request Handling

5.1 Handling Process

EV requests are generated by `ev_arrival_process(env)`. Each new EV triggers a complete decision and assignment pipeline implemented in `assign_ev_to_station(env, ev_id, ev_loc, ev_demand, reassignment=False)`. Conceptually, the handling process is:

1. **Generate EV and register it in results**

- A new EV gets an ID, arrival time, energy demand and location.
- A corresponding entry is created in `results['evs']`.

2. **Compute FCS options**

- `find_fcs_options(ev_loc, ev_demand)` evaluates all FCSs.
- For each FCS, it computes travel distance, travel cost, charging cost and expected waiting time based on current queue and service capacity.
- FCS options that violate the maximum allowed waiting time of 90 minutes are discarded.

3. **Compute existing MCV options**

- `find_mcv_options(ev_loc, ev_demand)` evaluates all currently deployed and available MCVs.
- Options respect MCV SOC constraints: if serving the EV would drop SOC below `minimum_capacity_mcv` the option is removed.
- Waiting time is derived from the number of EVs already in service and in the MCV's queue.

4. **Optionally compute “new MCV” options (Stackelberg only)**

- When `USE_GREEDY_MCV_DISPATCH = False`, `find_new_mcv_options` can propose the deployment of new MCVs at high rank meeting points.
- For each candidate meeting point, the function checks whether at least two EVs in the system would have strictly lower generalized cost with the new MCV compared to existing FCS options.
- If so, a “New MCV” option is added, linked to that meeting point and priced using the meeting point's dynamic price.

5. **Apply the EV decision rule**

- If no options remain after filtering, the EV is marked as not served via `mark_ev_not_served`.
- Under Stackelberg EV behavior, the EV selects the option with minimum C_{EV} .

- Under greedy EV behavior, the EV selects the option with minimum charging plus travel cost.

6. Start the charging process

- If the chosen option is an FCS, the EV starts `ev_process_fcs` which models queueing, charging, and enforcement of maximum waiting time.
- If the option is an existing MCV, the EV starts `ev_process_mcv` which includes queueing, charging and MCV SOC update.
- If the option involves a new MCV, a deployment process is created to move the MCV from the depot to the chosen meeting point before charging EVs there.

7. Reassignment of previously not served EVs

- The central deployment logic `update_mcv_deployment(env)` periodically rechecks EVs that were not served due to lack of capacity.
- When new MCV capacity becomes available, these EVs are reprocessed through `assign_ev_to_station` with `reassignment=True`.

6 MCV Behavior and Dynamic Deployment

6.1 Scenario based Deployment

MCV behavior combines several interacting mechanisms:

1. High utilization at FCSs

- FCS utilization and waiting time are logged every minute.
- If an FCS operates at or near 100 percent utilization for 10 consecutive minutes or if a short sliding window of FCS waiting times is consistently high, the system considers deploying one or more MCVs near that FCS.

2. Low utilization or idle MCVs

- The function `return_idle_mcv_to_depot(env)` finds MCVs that have had no allocations and no waiting EVs for more than 60 minutes and sends them back to the depot.
- The function `should_mcv_return` looks at MCV utilization over a configurable window and can also suggest returning under sustained low utilization.

3. Peak and off peak periods

- There are no hard coded time based rules for MCV deployment, but the higher arrival rate during early hours produces longer queues and more frequent trigger events that cause MCV deployment to high demand regions.
- During off peak periods, queues tend to be shorter, deployment triggers are less frequently satisfied and MCVs are more likely to return to the depot.

4. Battery SOC below threshold

- Any MCV whose SOC falls below `minimum_capacity_mcv` is treated as unable to accept new EVs.
- Such an MCV is marked as unavailable and eventually returned to the depot; this prevents over commitment of battery energy.

7 Dynamic Deployment Logic

7.1 Triggering Conditions

The central function `update_mcv_deployment(env)` aggregates several triggers and decides when and where to deploy new MCVs or to return under used ones to the depot. The triggers can be grouped as follows.

- **Short window waiting time check:**
 - `sliding_window_fcs_wait_time_check(current_time, window=5, wait_time_threshold=10)` inspects the last five minutes of FCS waiting time logs.
 - If the average waiting time across FCSs exceeds 10 minutes or if queues are already non empty in the first few minutes of the

simulation, it recommends deployment.

- **Continuous full utilization at an FCS:**

- `check_fcs_100_util_10mins(current_time)` checks whether any FCS has essentially been fully utilized for at least ten consecutive minutes.
- If such an FCS exists and MCVs are available, the system deploys MCVs near the most congested FCS.

- **Persistent queues at specific FCSs or meeting points:**

- `enough_waiting_evs_at_fcs_for_duration` tests if a given FCS has had at least a certain number of EVs waiting for a specified duration.
- `enough_waiting_evs_for_duration_meeting_point` provides a similar check at meeting points.
- These functions avoid reaction to short lived spikes and focus on sustained congestion.

- **Fifteen minute average utilization levels:**

- `check_avg_fcs_util_15mins` and `check_avg_mcv_util_15mins` compute average utilization for FCSs and MCVs over the last fifteen minutes.
- When FCS utilization is high and no MCVs are deployed, or when both FCS and MCV utilization are high, the system deploys additional MCVs near the busiest FCSs.

- **Demand still high check:**

- `demand_still_high` combines information about recent arrivals and not served EVs to decide if further MCV deployment is justified even after previous deployments.

- **Greedy MCV deployment baseline:**

- When `USE_GREEDY_MCV_DISPATCH = True`, the function `greedy_mcv_dispatch(env)` implements a simple rule:
 - * For each FCS, look at the most recent average waiting time from `fcs_minute_status`.

- * If the average is larger than a threshold GREEDY_WAIT_THRESHOLD_MIN and the cooldown period GREEDY_COOLDOWN_MIN since the last deployment for that FCS has passed, and an idle MCV is available, then deploy exactly one MCV near that FCS.
- Meeting point capacities and nearest meeting point selection are handled inside `deploy_mcv_near_fcs`.

7.2 Decision Flow

At each call to `update_mcv_deployment(env)` the following high level steps occur:

1. Return idle MCVs

- `return_idle_mcv_to_depot(env)` identifies MCVs that have been idle for at least 60 minutes and initiates `mcv_return_to_depot` processes.

2. Choose between Stackelberg or greedy deployment logic

- If `USE_GREEDY_MCV_DISPATCH = True`, then `greedy_mcv_dispatch(env)` is called and no Stackelberg multi-trigger deployment logic is executed.
- Otherwise, the following Stackelberg deployment triggers are evaluated.

3. Short-term reaction to queues

- The short window waiting time check is evaluated.
- If it indicates high average waiting time or a large number of EVs waiting, one or two MCVs are deployed to the busiest FCS that still has MCV capacity available in nearby meeting points.

4. Longer term utilization based deployment

- If the system has no MCVs deployed and the fifteen minute average FCS utilization exceeds a threshold, an MCV is deployed near the FCS with the longest queue.
- If both FCS and MCV utilization are high and enough MCVs are still idle at the depot, additional MCVs can be deployed.

5. Demand based refinement

- When long queues persist and demand is still classified as high, the code can deploy additional MCVs or redeploy under utilized MCVs closer to hotspots via `mcv_deployment` and related helper functions.

6. Reassigning not served EVs

- After deployments or returns, the algorithm rechecks EVs previously marked as not served when capacity was not available.
- These EVs are processed again through `assign_ev_to_station` using the updated set of MCV and FCS options.

8 Example Execution

8.1 Simulation Initialization

A typical run proceeds as follows:

- A base random seed `BASE_SEED = 44` is defined. An optional environment variable `SIM_RUN_INDEX` is read and added to `BASE_SEED` to create a run specific seed `run_seed`.
- The initial setup folder `Stackelberg Simple Simulation` is created if it does not exist.
- `InitialSetup_FCS.csv` is read using `robust_read_csv` to build `fcs_initial_df` and derive `fcs_locations` and `num_fcs`.
- `InitialSetup_MeetingPoints_Updated.csv` is read to construct dictionaries `meeting_points`, `meeting_point_prices`, and `meeting_point_ranks`.
- If `ADDITIONAL_FCS_COUNT > 0`, selected meeting points are promoted to FCSs and appended to `fcs_initial_df`.
- Optional EV data from the input `EV_Data.csv` and MCV initial states from `InitialSetup_MCV.csv` are loaded. Any EVs already assigned to MCVs at time zero are reconstructed in `results['evs']`.
- All global counters, dictionaries, and per hour statistics are initialized.

- A SimPy environment is created with `env = simpy.Environment()`.
- Processes are started for EV arrivals, MCV status monitoring, minute by minute FCS and MCV logging, and the three video animations.

8.2 Real Time Request Handling

During the simulation:

1. Every simulated minute, `ev_arrival_process` draws a Poisson number of EVs using `get_arrival_rate` and creates each EV with random location and energy demand.
2. For each EV, `assign_ev_to_station` computes cost based options across all FCSs and MCVs and then selects the minimum cost option based on the configured EV decision mode.
3. If the EV is assigned to an FCS, it joins the FCS resource queue and is served through `ev_process_fcs`, which logs waiting time, start time, and end time of charging, and updates the EV's record in `results`.
4. If the EV is assigned to an MCV, it joins the MCV's SimPy resource, experiences a waiting time, is charged, and triggers SOC updates and possible return to depot decisions through `ev_process_mcv`.
5. If no feasible option exists (for example because all FCS waiting times exceed the maximum and all MCVs are constrained by SOC), the EV is marked as not served and counts toward not served statistics for the current period.

8.3 Dynamic Deployment Example

To illustrate the deployment logic, consider a generic scenario:

- During the first two hours, high arrival rates produce significant queues at FCS 1.
- The five minute sliding average waiting time exceeds 10 minutes, and `sliding_window_fcs_wait_time_check` triggers deployment.
- `update_mcv_deployment` selects FCS 1 as the busiest station and calls `deploy_mcv_near_fcs`, which selects the nearest meeting point

that has capacity for additional MCVs and sends an MCV from the depot to that location.

- As MCVs start serving EVs near that meeting point, the FCS queues shorten. Later, as demand subsides and MCVs become under utilized, `return_idle_mcv_to_depot` sends them back to the depot.

This example is illustrative rather than tied to a specific numerical realization; actual deployments depend on random arrivals and the exact CSV inputs supplied.

9 Understand Implementation

In this section the code is mapped to the conceptual model so that readers can easily locate the relevant parts when extending or reconfiguring the simulation.

9.1 High Level Workflow of the Simulation

The overall workflow is:

1. Initialization phase

- Set global parameters such as simulation duration, number of MCVs, capacity and cost parameters (`max_num_mcv`, `battery_capacity_mcv`, `capacity_fcs`, `cost_per_km_ev`, `fixed_charging_price`, `c_mcv`, `c_fcs`, `zeta`, `delta`, `epsilon`, `cost_per_hour`, `charging_rate_mcv`, `charging_rate_fcs`, `max_wait_time_fcs`, `max_wait_time_mcv`).
- Configure Stackelberg versus greedy modes through the boolean flags: `USE_GREEDY_MCV_DISPATCH` and `USE_GREEDY_EV_SELECTION`.
- Load all CSV files using `robust_read_csv` and construct:
 - `fcs_initial_df`, `fcs_locations`, and `num_fcs`.
 - `meeting_points`, `meeting_point_prices`, and `meeting_point_ranks`.
 - `ev_data_df` for any pre existing EVs.

- `mcv_states` dictionary from `InitialSetup_MCV.csv` when available, including pre assigned EVs.
- Initialize the global `results` structure, which stores EV level, FCS level, MCV level, and aggregate statistics.
- Initialize per minute log lists `fcs_minute_status` and `mcv_minute_status` for utilization monitoring.
- Set up logging of terminal output to a file through the `Logger` class, which redirects `sys.stdout`.
- Instantiate the `SimPy` environment.

2. Process start phase

- Start EV arrivals: `env.process(ev_arrival_process(env))`.
- Start MCV status monitoring: `env.process(mcv_status_monitor(env))`.
- Start minute by minute logs for FCSs and MCVs: `env.process(log_fcs_status(env))` and `env.process(log_mcv_status(env))`.
- Start animation related processes or frame generation loops for three videos: EV arrivals, EV assignments, and full dynamics mapping.

3. Main simulation run

- `env.run(until=simulation_duration)` runs the environment for 480 minutes.
- During the run:
 - (a) EVs are generated each minute and passed to `assign_ev_to_station`.
 - (b) EVs either start `ev_process_fcs` or `ev_process_mcv` if assigned, otherwise they are recorded as not served.
 - (c) MCV utilization, SOC, and queues are periodically evaluated, and `update_mcv_deployment` is called from the EV arrival process to manage deployment and returns.
 - (d) Per minute logging processes update utilization logs and provide input for both Stackelberg and greedy deployment poli-

cies.

4. Post processing and output

- After the simulation ends, the script computes leader profit, utilization statistics, and summarizes EV service outcomes.
- It writes multiple CSV files in an output folder whose name is based on `base_output_folder` and an optional environment variable `SIM_RUN_TAG`.
- It generates static figures of the city map at selected times and writes three MP4 videos if `ffmpeg` is available.
- A final summary of key statistics is printed to the terminal log and the log file is closed.

9.2 Function by Function Breakdown

This subsection groups functions by role. The exact function names are as in the Python script.

A. Data and configuration utilities

- `robust_read_csv(filepath, default_df, required_cols=None)`: central CSV loader with error handling and default fallbacks.
- `generate_ev_location()`: samples a random EV location within the simulation area.
- `calculate_road_distance(loc1, loc2)`: returns road distance by multiplying Euclidean distance with `road_distance_factor`.
- `get_arrival_rate(current_time)`: returns the time dependent Poisson arrival rate for EVs.

B. EV arrival and assignment

- `ev_arrival_process(env)`: main process that creates EVs every minute, updates per hour counters, and triggers MCV deployment updates after each minute.

- `assign_ev_to_station(env, ev_id, ev_loc, ev_demand, reassignment=False)`: coordinates the creation of cost based options for FCSs, existing MCVs, and new MCVs; applies the EV decision rule; and dispatches EVs to `ev_process_fcs` or `ev_process_mcv`.
- `find_fcs_options(ev_loc, ev_demand)`, `find_mcv_options(ev_loc, ev_demand)`, `find_new_mcv_options(env, ev_id, ev_loc, ev_demand)`: helpers that compute feasible options and their generalized costs.
- `mark_ev_not_served(env, ev_id, ev_demand, reassignment)`: records EVs that cannot be served by any option, keeping track of period specific not served demand.

C. Charging processes

- `ev_process_fcs(env, ev_id, ev_demand, chosen_fcs, results)`: SimPy process that enforces maximum waiting time at FCSs, updates waiting and charging times, and records per EV metrics.
- `ev_process_mcv(env, ev_id, ev_demand, chosen_mcv_id, results)`: similar to `ev_process_fcs` but for MCVs; updates MCV SOC, tracks allocations, and can trigger MCV returns.

D. MCV deployment and return

- `deploy_mcv_near_fcs(env, fcs_id)`: selects an appropriate meeting point near an FCS and deploys an MCV if available and the meeting point has capacity for additional MCVs.
- `update_mcv_deployment(env)`: central function combining all triggers and performing MCV returns and new deployments according to Stackelberg or greedy logic.
- `return_idle_mcv_to_depot(env)`, `mcv_return_to_depot(env, mcv_id)`: move under utilized or idle MCVs back to the depot.
- `should_mcv_return(mcv, current_time, duration, utilization_threshold)`: determines whether an MCV should return based on its recent utilization history.

E. Monitoring and logging

- `update_mcv_status(current_time)`: updates internal status of each MCV and stores entries in `results[ 'mcv_status_log' ]`.
- `mcv_status_monitor(env)`: runs periodically to keep MCV status consistent with utilization and SOC conditions.
- `log_fcs_status(env)` and `log_mcv_status(env)`: record minute by minute utilization statistics into lists that are later converted into `FCS_Minute_Status.csv` and `MCV_Minute_Status.csv`.
- `calculate_leaders_profit(results)`: computes revenue and costs as described earlier.

F. Visualization and animation

- `plot_simulation_state(ax, t, draw_lines=False)`: generates a snapshot of the system at minute t , including depot, FCSs, meeting points, MCVs, and EVs.
- `plot_video1_state(t)`, `plot_video2_state(t)`, `plot_video3_state(t)`: generate frames for the three MP4 videos: EV arrivals over time, EV assignments, and full dynamics including queues and deployment.
- Static figures at selected times (for example minute 0, 180, 360, 480) are written as PNG files to the output folder for documentation in the paper.

G. Output generation

- EV level results are written to `EV_Data.csv` in the output folder, including demand, chosen source, costs, waiting and charging times, and spatial coordinates.
- FCS level summary is written to `FCS_Data.csv`, including number of EVs served, total energy dispensed, and utilization measures.
- MCV level summary is written to `MCV_Data.csv`, including EVs served, total energy sold, prices, and utilization.

- Minute level FCS and MCV logs are written as `FCS_Minute_Status.csv` and `MCV_Minute_Status.csv` respectively.
- Hourly utilization statistics are written to `FCS_Utilization_Hourly.csv` and `MCV_Utilization_Hourly.csv`, capturing average utilization per hour for FCSs and MCVs.
- A per hour summary of demand, served energy, and not served energy is written to `Hourly_Summary.csv`.
- A detailed MCV status history is written to `MCV_Status_Log.csv`, which records key state variables for each MCV over time.
- An overall summary CSV and human readable terminal log are produced to facilitate quick inspection of each run.

9.3 Simulation Start to End: Overview

For quick reference, the life cycle of a simulation run is:

1. Configure global parameters and CSV inputs.
2. Run the Python script with a given `SIM_RUN_INDEX` and, optionally, `SIM_RUN_TAG` to select the random seed and output folder naming.
3. The script initializes FCSs, meeting points, EVs, and MCVs, starts all SimPy processes, and executes `env.run(until=480)`.
4. During the run, EVs arrive, choose between FCSs and MCVs, and are served or marked as not served; MCVs are deployed, moved, and returned according to Stackelberg or greedy rules.
5. After 480 minutes, the script aggregates results, writes CSV outputs and figures, generates videos, prints a summary, and closes the log.

9.4 Simulation at a Glance

To change or extend the experiment, it is sufficient to understand where to modify the following:

1. **Scenario parameters:** modify global variables near the top of the script (number of MCVs, charging rates, maximum waiting times, cost coefficients, additional FCS count).

2. **Behavioral mode:** set `USE_GREEDY_MCV_DISPATCH` and `USE_GREEDY_EV_SELECTION` to choose among CASE A to CASE D; set `max_num_mcv = 0` to obtain CASE E.
3. **Spatial and price configuration:** edit the CSV files in `StackelbergSimpleSimulation` to change FCS coordinates, meeting point locations, dynamic prices, and initial MCV placements.
4. **Demand profile:** adapt `get_arrival_rate` if a different time of day arrival pattern is desired, or change the demand distribution for EV energy requirements.
5. **Output and logging:** adjust the output folder base name and the environment variables to organize experiments; use the generated CSVs and videos to validate that the implementation matches the conceptual model described in the paper.